Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

**Лабораторная работа №4**

По дисциплине: «АПО ЭВМиС»

Тема: «Архитектура и программирование сопроцессора»

Выполнил:
Студент 4-го курса
Группы АС-66
Савинец М. Д.
Проверил:
Кулик А.Д.

Брест 2026

## Цель работы

Изучить архитектуру и программную модель сопроцессора, применить его целочисленные и вещественные команды при вычислении функции.

## Вариант 6

Заданы два входных значения x и y. Требуется вычислить результат по одной из трёх ветвей:

$r = 5x + 2xy + 1$, если $x + y > 9$;

$r = (2xy + 3) / (y^2 + 4)$, если $x + y < -5$;

$r = 3x^2 - 2y^2 + 1$, если $-5 \le x + y \le 9$.

В таблице методички буквой y обозначены одновременно вход и результат. Здесь результат назван r, чтобы не смешивать его с введённым значением y. Точки $x + y = -5$ и $x + y = 9$ относятся к третьей ветви.

## Среда выполнения и организация программы

Программа собрана на macOS с процессором Apple Silicon как исполняемый файл Intel x86-64. Запуск на этом компьютере выполнен через Rosetta 2: она транслирует инструкции Intel на платформу Apple Silicon; это не прямой запуск на процессоре Intel. В ассемблерном файле используются инструкции сопроцессора x87. Ввод и вывод организованы на C++. Вопрос допустимости такой среды для сдачи следует отдельно согласовать с преподавателем.

Обе функции принимают x и y от −1000 до 1000. В режиме integer входы целые; в режиме real — конечные вещественные числа. Ограничение выбрано в реализации, а не задано в методичке. Обе функции возвращают double: дробный результат целочисленного раздела не отбрасывается.

## Раздел 1. Целочисленные команды сопроцессора

Функция integer получает целые x и y. Сумма x + y сравнивается с границами средствами процессора; затем вызывается внутренняя процедура соответствующей ветви. Для расчёта выражений используются команды x87 fildl, fimull, fiaddl, fisubl, fidivl с целочисленными операндами в памяти. В ветви с делением знаменатель $y^2 + 4$ положителен, а результат может быть дробным. Например, при $x = -6$ и $y = 0$ он равен 0,75.

## Раздел 2. Вещественные команды сопроцессора

Функция real принимает два значения double. Сумма x + y сначала вычисляется инструкцией SSE. Если округлённая сумма попадает точно на границу, алгоритм TwoSum восстанавливает остаток, чтобы правильно выбрать ветвь для представленных входов. Вычисление каждой ветви выполнено в отдельной внутренней процедуре с командами x87 fldl, fmull, faddl, fsubl и fdivl. Вещественные операции и сохранение результата могут сопровождаться округлением. Такой выбор ветви предполагает стандартное округление к ближайшему и включённые субнормальные числа.

Каждая внутренняя процедура оставляет результат на вершине стека x87. Внешняя функция снимает его в память и помещает в регистр xmm0 согласно соглашению вызовов x86-64, чтобы вернуть результат программе на C++.

## Проверка программы

Программа собрана и проверена на данном компьютере: 20035 численных случаев, 26 проверок у границ, 14548 проверок с независимым рациональным эталоном, 16 корректных и 21 некорректный консольный ввод, а также 7 проверок ввода ненулевых субнормальных чисел и 2 проверки потери значения до нуля. Проверены все три ветви и точное равенство граничным значениям. Результаты относятся к этому запуску, а не к оценке преподавателя.

| Режим | x | y | Результат r |
|---|---|---|---|
| integer | 10 | 0 | 51 |
| integer | −6 | 0 | 0,75 |
| integer | −5 | 0 | 76 |
| integer | 9 | 0 | 244 |
| real | −8,25 | 1,5 | −3,48 |
| real | 5,5 | 4,25 | 75,25 |
| real | −2,5 | −2,5 | 7,25 |

Таблица содержит результаты реальных запусков. Ввод десятичных чисел в программу выполняется через точку.

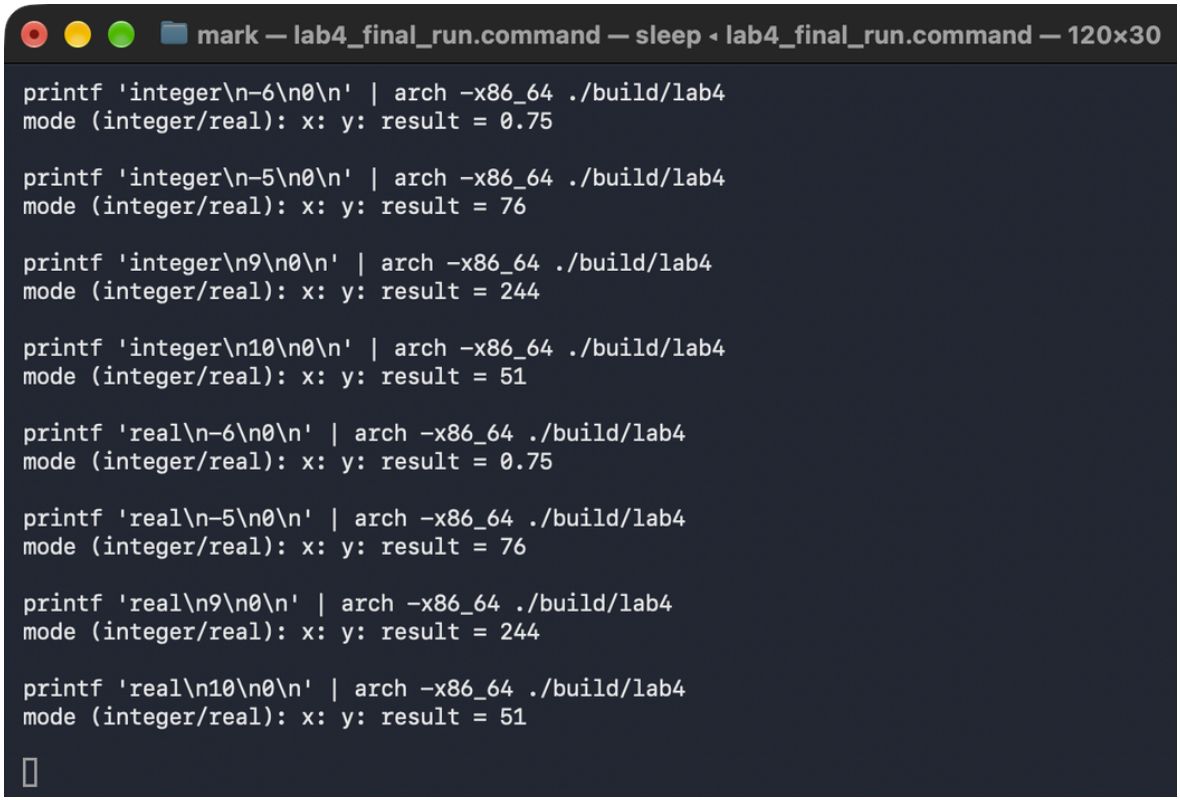



Рисунок 1 — Проверка программы в терминале

## Текст программы для обоих разделов

Ниже приведены полные листинги основной программы, ассемблерных функций и сценария сборки. Дополнительные тестовые файлы находятся в каталоге проекта.

### main.cpp

```
#include <cerrno>
#include <cctype>
#include <cmath>
#include <cstdlib>
#include <iomanip>
#include <iostream>
#include <limits>
```

```cpp
#include <sstream>
#include <string>

extern "C" double integer(int x, int y);
extern "C" double real(double x, double y);

// One complete numeric token per line: reject trailing garbage and extra tokens.
template <typename T>
bool read_value(const char* prompt, T& value) {
    std::cout << prompt << std::flush;
    std::string line;
    if (!std::getline(std::cin, line)) return false;
    std::istringstream input(line);
    if (!(input >> value)) return false;
    input >> std::ws;
    return input.eof();
}

// strtod may set ERANGE for a valid nonzero subnormal: keep that value.
// Reject only underflow all the way to zero; literal zero remains valid.
// The integer overload above retains its original strict parsing and range checks.
bool read_value(const char* prompt, double& value) {
    std::cout << prompt << std::flush;
    std::string line;
    if (!std::getline(std::cin, line)) return false;
    const char* start = line.c_str();
    char* end = nullptr;
    errno = 0;
    value = std::strtod(start, &end);
    if (end == start || (errno == ERANGE && value == 0.0)) return false;
    while (*end && std::isspace(static_cast<unsigned char>(*end))) ++end;
    // Checking against size also rejects an embedded NUL followed by extra data.
    return end == start + line.size();
}

int main() {
    std::cout << "mode (integer/real): " << std::flush;
    std::string mode;
    if (!std::getline(std::cin, mode) || (mode != "integer" && mode != "real")) {
        std::cerr << "error: mode must be integer or real\n";
        return 1;
    }
    double result;
    if (mode == "integer") {
        int x, y;
        if (!read_value("x: ", x) || !read_value("y: ", y) ||
            x < -1000 || x > 1000 || y < -1000 || y > 1000) {
            std::cerr << "error: x and y must be integers in [-1000, 1000]\n";
            return 1;
        }
        result = integer(x, y);
    } else {
        double x, y;
        if (!read_value("x: ", x) || !read_value("y: ", y) ||
            !std::isfinite(x) || !std::isfinite(y) ||
            std::fabs(x) > 1000 || std::fabs(y) > 1000) {
            std::cerr << "error: x and y must be finite real numbers in [-1000, 1000]\n";
            return 1;
        }
        result = real(x, y);
    }
    std::cout << "result = " << std::setprecision(std::numeric_limits<double>::max_digits10)
              << result << '\n';
}
```

# variant6.S

```asm
# macOS x86-64 SysV ABI: int x/y in edi/esi, result double in xmm0.
# Preconditions: -1000 <= x,y <= 1000. Every branch leaves one x87 value.
.text
.globl _integer
_integer:
    pushq %rbp
    movq %rsp, %rbp
    subq $48, %rsp
    movl %edi, -4(%rbp)
    movl %esi, -8(%rbp)
    movl $1, -12(%rbp)
```

```
    movl $2, -16(%rbp)
    movl $3, -20(%rbp)
    movl $4, -24(%rbp)
    movl $5, -28(%rbp)
    leal (%rdi,%rsi), %eax
    cmpl $9, %eax
    jg Lint_high_call
    cmpl $-5, %eax
    jl Lint_low_call
    call Lint_middle
    jmp Lint_return
Lint_high_call:
    call Lint_high
    jmp Lint_return
Lint_low_call:
    call Lint_low
Lint_return:
    fstpl -48(%rbp)       # pop; SysV returns double in xmm0, not st(0)
    movsd -48(%rbp), %xmm0
    leave
    ret


# Internal procedures use the parent's rbp frame and return a single st(0).
Lint_high:                # 5*x + 2*x*y + 1
    fildl -4(%rbp)
    fimull -8(%rbp)
    fimull -16(%rbp)
    fistpl -32(%rbp)
    fildl -4(%rbp)
    fimull -28(%rbp)
    fiaddl -32(%rbp)
    fiaddl -12(%rbp)
    ret
Lint_low:                 # (2*x*y + 3)/(y*y + 4)
    fildl -8(%rbp)
    fimull -8(%rbp)
    fiaddl -24(%rbp)
    fistpl -32(%rbp)
    fildl -4(%rbp)
    fimull -8(%rbp)
    fimull -16(%rbp)
    fiaddl -20(%rbp)
    fidivl -32(%rbp)
    ret
Lint_middle:              # 3*x*x - 2*y*y + 1
    fildl -8(%rbp)
    fimull -8(%rbp)
    fimull -16(%rbp)
    fistpl -32(%rbp)
    fildl -4(%rbp)
    fimull -4(%rbp)
    fimull -20(%rbp)
    fisubl -32(%rbp)
    fiaddl -12(%rbp)
    ret


# double x/y in xmm0/xmm1. Compare the EXACT sum of the two input doubles.
# Assumes the usual ABI environment: round-to-nearest, gradual underflow,
# and SSE DAZ/FTZ disabled; see README for the TwoSum proof.
.globl _real
_real:
    pushq %rbp
    movq %rsp, %rbp
    subq $32, %rsp
    movsd %xmm0, -8(%rbp)
    movsd %xmm1, -16(%rbp)
    addsd %xmm1, %xmm0          # s = round(x + y)
    comisd Lnine(%rip), %xmm0
    ja Lreal_high_call
    je Lreal_tie_high
    comisd Lminusfive(%rip), %xmm0
    jb Lreal_low_call
    je Lreal_tie_low
Lreal_middle_call:
    call Lreal_middle
    jmp Lreal_return
Lreal_tie_high:
    movl $1, %eax              # rounded sum was 9
    jmp Lreal_residual
```

```
Lreal_tie_low:
    xorl %eax, %eax             # rounded sum was -5
Lreal_residual:
    # Knuth TwoSum: b_virtual = s-x, a_virtual = s-b_virtual;
    # residual = (x-a_virtual) + (y-b_virtual), exactly x+y-s.
    # Do not replace with long double: even 80 bits lose DBL_TRUE_MIN at 9.
    movapd %xmm0, %xmm2
    subsd -8(%rbp), %xmm2       # b_virtual
    movapd %xmm0, %xmm3
    subsd %xmm2, %xmm3          # a_virtual
    movsd -8(%rbp), %xmm4
    subsd %xmm3, %xmm4          # x-a_virtual
    subsd %xmm2, %xmm1          # y-b_virtual
    addsd %xmm1, %xmm4          # exact residual (including subnormals)
    pxor %xmm5, %xmm5
    testl %eax, %eax
    jz Lreal_residual_low
    comisd %xmm5, %xmm4
    ja Lreal_high_call           # 9 + positive error > 9
    jmp Lreal_middle_call
Lreal_residual_low:
    comisd %xmm5, %xmm4
    jb Lreal_low_call            # -5 + negative error < -5
    jmp Lreal_middle_call
Lreal_high_call:
    call Lreal_high
    jmp Lreal_return
Lreal_low_call:
    call Lreal_low
Lreal_return:
    fstpl -24(%rbp)
    movsd -24(%rbp), %xmm0
    leave
    ret
Lreal_high:               # 5*x + 2*x*y + 1
    fldl -8(%rbp)
    fmull -16(%rbp)
    fmull Ltwo(%rip)
    fstpl -24(%rbp)
    fldl -8(%rbp)
    fmull Lfive(%rip)
    faddl -24(%rbp)
    faddl Lone(%rip)
    ret
Lreal_low:                # (2*x*y + 3)/(y*y + 4)
    fldl -16(%rbp)
    fmull -16(%rbp)
    faddl Lfour(%rip)
    fstpl -24(%rbp)
    fldl -8(%rbp)
    fmull -16(%rbp)
    fmull Ltwo(%rip)
    faddl Lthree(%rip)
    fdivl -24(%rbp)
    ret
Lreal_middle:             # 3*x*x - 2*y*y + 1
    fldl -16(%rbp)
    fmull -16(%rbp)
    fmull Ltwo(%rip)
    fstpl -24(%rbp)
    fldl -8(%rbp)
    fmull -8(%rbp)
    fmull Lthree(%rip)
    fsubl -24(%rbp)
    faddl Lone(%rip)
    ret


.section __TEXT,__const
.p2align 3
Lone:       .double 1.0
Ltwo:       .double 2.0
Lthree:     .double 3.0
Lfour:      .double 4.0
Lfive:      .double 5.0
Lnine:      .double 9.0
Lminusfive: .double -5.0
```

## run_mac.command

```
#!/bin/bash
```

```
# Build and test the x86-64 binaries; run interactively unless --test is given.
set -euo pipefail
cd "$(dirname "$0")"
if [[ "${1:-}" != "" && "${1:-}" != "--test" ]] || [[ $# -gt 1 ]]; then
    printf 'Usage: %s [--test]\n' "$0" >&2
    exit 2
fi
if [[ "$(uname -s)" != Darwin ]]; then
    printf 'error: macOS is required\n' >&2
    exit 1
fi
mkdir -p build
flags=(-arch x86_64 -std=c++17 -O2 -Wall -Wextra -Werror)
clang++ "${flags[@]}" main.cpp variant6.S -o build/lab4
clang++ "${flags[@]}" tests.cpp variant6.S -o build/tests
clang++ "${flags[@]}" test_exact_sum.cpp variant6.S -o build/test_exact_sum
clang++ "${flags[@]}" test_fraction_bridge.cpp variant6.S -o build/test_fraction_bridge
arch -x86_64 build/tests
arch -x86_64 build/test_exact_sum
python3 test_fraction.py "$PWD/build/test_fraction_bridge"
python3 test_cli.py "$PWD/build/lab4"
python3 test_subnormal_cli.py "$PWD/build/lab4"
if [[ "${1:-}" != "--test" ]]; then
    exec arch -x86_64 build/lab4
fi
```

## Вывод

Для варианта 6 реализованы и проверены две функции с целочисленными и вещественными командами сопроцессора x87. Для каждого условия используется отдельная внутренняя процедура; проверены граничные значения и дробный результат целочисленного режима.